Gebeta v1: Tech Stack and Timeline

Based on the Gebeta v1 Rules Spec (Nov 2, 2026) · Team of 3 · Target: 12 weeks

1. Overview

Gebeta v1 is a mobile version of the traditional Ethiopian game. Version one includes:

- A machine-learning opponent trained to play Gebeta
- Single player with three difficulty levels: Novice, Amateur, Master
- Two-player mode on one phone

The spec shapes the technical choices. The state is tiny (12 holes, 2 pits, 6 actions per move), the game is deterministic with perfect information, and a single turn can be a long relay chain. This means the AI can be small, the engine must be fast, and the Dart and Python engines must behave identically.

2. Tech Stack

2.1 Mobile app

Need	Choice	Why
Framework	Flutter (Dart)	One codebase for Android and iOS
Game rendering	Flame	Good for 2D board and seed animations
State management	Riverpod	Simple, testable game state
Local storage	Hive or shared_preferences	Settings, stats, saved game
Audio	flame_audio	Seed drop and capture sounds
Localization	Flutter intl	Amharic and English from day one
Background work	Dart isolates	Run AI search without freezing the UI

2.2 Game engine (core of the project)

- A **Dart engine** (used by the app) and a **Python engine** (used for training), both written from the rules spec.

- One **shared test file** (`test_cases.json`) that both engines must pass in CI. Start with the six worked examples from the spec, then add:
  - Player 1 mirror cases (opposite hole is 11 − i)
  - A hole with 13+ seeds (lap over the start hole)
  - A loop-abort case taken from the cycling positions in the spec simulation
  - Game-end cases (no seeds on the mover's side, leftover seeds, draw)
- **Loop safeguard:** a cap of 1,000 sowings per turn in v1. Per-turn cycle detection can come later.
- **Whole-game cap:** about 300 turns, then adjudicate by pit counts, so self-play never hangs.

2.3 AI and machine learning

Need	Choice
Language	Python
Libraries	PyTorch, NumPy (Numba if self-play is slow)
Baseline opponent	Minimax with alpha-beta pruning
Training method	AlphaZero-style self-play with a small MLP (policy and value heads) and MCTS
Input encoding	12 hole counts (scaled), both pit counts; always from the current player's perspective (holes renumbered 0-5), so one network plays both seats
Reward	Win +1, draw 0, loss −1
Experiment tracking	TensorBoard
Compute	Team laptops, or free Google Colab / Kaggle GPU
Deployment	Export weights as JSON or binary and run inference in plain Dart (no native plugins)

The minimax baseline comes first. It benchmarks the trained model, can serve Novice and Amateur, and is the fallback for Master.

2.4 Difficulty levels

Level	Implementation
Novice	Greedy or random moves with a 20-30% deliberate mistake rate
Amateur	Shallow minimax, or an early training checkpoint without MCTS
Master	Best network with MCTS (or deep alpha-beta if it performs better)

Validate with tournaments: Master should beat Amateur, and Amateur should beat Novice, by clear margins (about 80% or more).

2.5 Multiplayer on one phone

Uses the same engine with no AI. The board is rotated or mirrored so each player can sit on their own side. Build this first, since it only needs the engine and the board UI.

2.6 Tooling and release

- **Source control and CI:** GitHub with GitHub Actions (runs the shared engine tests on every push)
- **Design:** Figma
- **Monitoring (optional):** Firebase Crashlytics and Analytics
- **Distribution:** Google Play Console (and an Apple Developer account if iOS is included)
- **Store requirements:** privacy policy page, screenshots, store listing
- **Backend:** none needed in v1

3. Team Roles

Role	Responsibilities
Dev A (ML)	Python engine, minimax baseline, self-play training, evaluation, model export

Role	Responsibilities
Dev B (App)	Flutter project, Dart engine, game logic, AI inference integration, performance
Dev C (UI/UX and QA)	Design, animations, sound, localization, rules/tutorial screen, testing, store listing

4. Timeline (12 weeks)

Week	Dev A (ML)	Dev B (App)	Dev C (UI/QA)
1	Python engine, shared test JSON	Flutter setup, CI	Wireframes; lock the "Proposed" rules
2-3	Minimax baseline, tournament harness	Dart engine passes shared tests, board UI	Seed/sowing animations, sound
3		**Same-phone multiplayer playable**	Player 1 mirror tests
4-7	Self-play training, tuning, evaluation vs minimax	Menus, settings, difficulty selector, save/resume	Amharic/English text, rules and tutorial screen
8	Export model	Integrate inference in an isolate; **first single-player build**	Playtest with people who know Gebeta
9	Difficulty calibration	Performance on low-end phones	Bug triage
10	Final model	Polish, accessibility	Store listing
11			Closed beta, crash reporting
12		Release build	Release

5. Risks and Mitigations

Risk	Mitigation
ML training does not reach the target strength by week 8	Ship Master as deep alpha-beta; add the trained model in v1.1
Dart and Python engines disagree, so the AI plays a different game than the app	Shared test file run in CI on every push
Proposed rules (end condition, leftover seeds, draw, first player) change late	Lock them in week 1; the training reward depends on them
Rare relay loops or endless games hang training or the app	1,000-sowing turn cap and a 300-turn game cap
Poor performance on cheap Android phones	Test on low-end devices early; run AI in an isolate; keep the model tiny
Players know different regional variants	In-app rules and tutorial screen

6. Immediate Next Steps

1. Hold a rules meeting and confirm the Proposed items.
2. Create the repo, CI, and the shared `test_cases.json` with the six spec examples plus Player 1 mirrors.
3. Build the Python engine and the minimax baseline.
4. Build the Dart engine and the board UI, then same-phone multiplayer.